Hollow ArtPass Asset Intake Guide

Blender now, Substance Painter later - practical replacement steps for Rafal and Martin

Generated 2026-05-01. Scope: production/prototype visuals for existing Hollow ArtPass roles. Runtime gameplay remains code/data-authoritative.

The One Rule

Every production asset becomes a visual child inside an existing active **AP_*** or **VFX_*** prefab. The prefab root keeps the presentation role marker. The model/material can change; gameplay collision, scripts, room data, damage, rewards, and traversal do not move into the art prefab.

- Rafal creates clean FBX models, textures, and eventually Substance texture sets.

- Martin imports them into the intake folder, edits the matching AP_* wrapper prefab, and validates in Developer Lab plus debug spawn.

- If a model looks correct in the Unity asset preview but wrong in game, first check unapplied Blender transforms and prefab child scale/origin.

What Went Wrong With The Chest

The chest FBX preview was valid, but the actual mesh inside the FBX was a cube and the long chest shape lived on the FBX object transform: scale roughly X 1, Y 0.642, Z 1.951. Our wrapper referenced the raw mesh directly, so Unity lost that transform and the chest turned into a cube/flat/invisible-looking preview depending on placement.

Fix pattern: in Blender, apply transforms before export. In Unity, keep the AP wrapper at scale 1 and only use child transform as a deliberate calibration layer. Do not rely on hidden FBX object scale for the final shape.

Recommended Folder Contract

Purpose	Path / Naming
Raw incoming asset batch	Assets/_Hollow/Art/Intake/Rafal/M##_or_Date///
Active runtime visual prefab	Assets/_Hollow/Prefabs/ArtPass/AP_.prefab or VFX_.prefab
Project Unity materials	Assets/_Hollow/Art/Materials/ArtPass/AP_M_.mat
Textures	Keep near intake asset at first; later move accepted production textures to a stable Art/Textures folder if desired.
Validation route	Developer Lab scene + in-game Developer Lab + bottom-right debug spawn menu.

Example chest intake folder:

```
Assets/_Hollow/Art/Intake/Rafal/M38/items/Chest/
Chest.fbx
chest.jpg
optional notes.txt
```

Blender Export Checklist

Use this before every FBX export. This is the cheapest place to prevent 90 percent of Unity import pain.

- Set units to Metric. Treat 1 Blender unit as 1 Unity meter.

- Model with Y as vertical height. Unity gameplay floor uses X/Z as floor plane and Y as up.

- Place the object center on X/Z origin. Put the bottom of the visible model at Y = 0 unless the object intentionally floats.

- Apply transforms before export: Object Mode, select model, Ctrl-A / Apply Rotation and Scale. After applying, object scale should read 1, 1, 1.

- Do not leave the real shape in object scale. If a chest is long, the mesh vertices should be long after Apply Scale.

- Remove gameplay collision objects unless they are clearly named visual-only and will not be used by runtime. Gameplay collision is authored in code/room data.

- Delete export cameras and lights unless the asset is intentionally a VFX/light prefab and Martin has requested it.

- Use clear object names: Model, Lid, Latch, Gem, Cloth, Blade, etc. Avoid Cube.001 for final handoff.

- Keep material slot names useful: M_ChestWood, M_MetalBand, M_Glow, M_EnemyShell.

- Before export, check dimensions in Blender Item panel. Write the intended Unity dimensions in a notes file if it matters.

FBX export settings from Blender

Setting	Value / Rule
Format	FBX binary
Selected Objects	On. Export only the intended asset hierarchy.
Apply Transform	On when available. Still apply rotation/scale manually first.
Forward / Up	Use Unity-friendly orientation. If unsure, export test with -Z Forward, Y Up, then verify in Unity.
Scale	1.0. Do not use export scale to fix object size.
Add Leaf Bones	Off for static props. For rigged characters, coordinate separately.
Bake Animation	Off unless animation is intentionally included.
Path Mode	Copy or Auto is acceptable for prototype, but do not depend on embedded/absolute texture paths for final.

Texture And Material Naming

Asset Type	Recommended Names
FBX mesh	chest_normal.fbx, enemy_charger.fbx, rock_obstacle.fbx
Base color texture	T_ChestNormal_BaseColor.png or chest_normal_basecolor.png
Normal map	T_ChestNormal_Normal.png
Metallic/smoothness	T_ChestNormal_MetallicSmoothness.png or clearly named separate maps
Unity material	AP_M_ChestBasic.mat, AP_M_EnemyCharger.mat
Prefab wrapper	AP_ChestBasic.prefab, AP_EnemyCharger.prefab

Unity Replacement Workflow A: Fast Prototype

Use this when Rafal gives one FBX and one texture, and we just need to see it in game quickly.

- Copy the FBX and texture into the intake folder, for example Assets/_Hollow/Art/Intake/Rafal/M##_or_Date/items/Chest/.

- Select the FBX in Unity. In Model import settings: Scale Factor 1, Generate Colliders off, Import Cameras off, Import Lights off for normal props.

- Check the Unity preview. If it looks wrong there, fix Blender export first.

- Create or update a Unity material in Assets/_Hollow/Art/Materials/ArtPass/. Use Universal Render Pipeline/Lit and assign the texture to Base Map.

- Open the matching active prefab under Assets/_Hollow/Prefabs/ArtPass/. Do not create a new runtime prefab unless a new PresentationPrefabRole is being added.

- Keep the root named AP_. Keep PresentationVisualMarker on the root with the correct role and fallback unchecked.

- Delete the old placeholder child. Drag the FBX model or mesh child into the AP prefab root as a visual child.

- Assign the Unity material to every MeshRenderer on the model child.

- Set child local position/rotation/scale. Preferred: position 0,0,0; rotation 0,0,0; scale 1,1,1. If calibration is needed, document it.

- Save prefab. Test in Developer Lab authoring scene, in-game Developer Lab, and debug spawn menu.

Unity Replacement Workflow B: Safer Production Path

Use this once assets are more final, or when Substance Painter textures are involved.

- Treat the FBX as mesh/hierarchy only. Keep textures as separate PNG/TGA files, not as hidden embedded dependencies.

- Create explicit Unity materials for each material slot. Assign BaseColor, Normal, Metallic/Smoothness, Emission as needed.

- If the FBX contains multiple child meshes, keep them under one AP wrapper root. Do not add gameplay scripts or colliders to those children.

- For animated assets later, keep Animator/Animation only when intentionally requested. Basic static ArtPass props should not own gameplay timing.

- If a child needs glow/VFX, add visual-only particles/lights under the AP prefab. Confirm they are safe on VisionOS and not expensive.

- Record any intentional offsets/scales in the prefab notes or issue comment so future validation does not treat them as mistakes.

Substance Painter Later

- Use one texture set per logical asset where possible. Keep names aligned with the Unity role or asset id.

- Export Unity URP-friendly maps: BaseColor, Normal, Metallic/Smoothness, AO, Emission if needed.

- Use power-of-two sizes where practical: 512 for tiny pickups/coins, 1024 for enemies/props, 2048 only for hero/boss assets if justified.

- Avoid extremely high roughness/black albedo combinations that disappear in the dark prototype lighting.

- Keep a simple color-readability rule: enemies, pickups, keys, portals, and hazards must read instantly from the gameplay camera.

Prefab Wrapper Contract

This is the Unity-side rule that lets us swap art without breaking gameplay.

Wrapper Part	Requirement
Root GameObject	Name stays AP_ or VFX_. Transform ideally 0 position, identity rotation, scale 1.
PresentationVisualMarker	Must exist on root. Role must match the catalog binding. Fallback unchecked for production/prototype art.
Visual children	Meshes, renderers, particle systems, line renderers, lights only when intentionally visual.
Forbidden on visuals	Gameplay colliders, rigidbodies, player/enemy/chest/reward controllers, traversal scripts, save-state scripts.
Bounds	Visible at scale 1, centered on X/Z origin, bottom at Y = 0 for floor-standing objects.
Materials	No missing magenta. No broken external absolute paths. Textures must live in the Unity project.
Runtime source of truth	Gameplay systems instantiate by PresentationPrefabRole through PresentationPrefabResolver.

Critical sanity check

If the FBX preview looks good but the AP prefab looks wrong, check whether the FBX model transform contains scale/rotation that was not applied in Blender. The AP wrapper must preserve the final mesh proportions either by applied mesh vertices or a deliberate child calibration scale.

Acceptance Checklist For Every Asset

[ ] FBX opens in Unity preview and has the expected shape/proportions.

[ ] Model import Scale Factor is 1. Generate Colliders is off unless specifically requested for an editor-only preview.

[ ] FBX has no unwanted cameras/lights for normal props.

[ ] All visible meshes have materials assigned.

[ ] Textures are inside Assets/_Hollow and not only referenced from Rafal local disk or Blender cache.

[ ] AP wrapper root has PresentationVisualMarker with correct role.

[ ] AP wrapper root scale is 1,1,1 unless there is a documented exception.

[ ] Visual child bottom sits at Y = 0 for grounded props.

[ ] No gameplay scripts, colliders, or rigidbodies exist under the visual child.

[ ] Developer Lab authoring scene shows the object.

[ ] In-game Developer Lab or debug spawn shows the same object and proportions.

Current Hollow ArtPass Role Map

Role	Active Prefab	Suggested Intake Folder	Notes
Player	AP_Player.prefab	characters/player_body	1.5m-1.8m readable body, bottom at floor.
EnemyNormal	AP_EnemyNormal.prefab	enemies/enemy_normal	Small combat silhouette.
EnemyFlying	AP_EnemyFlying.prefab	enemies/enemy_flying	Can float visually; no collider authority.
EnemyFast	AP_EnemyFast.prefab	enemies/enemy_fast	Fast enemy silhouette.
EnemyHeavy	AP_EnemyHeavy.prefab	enemies/enemy_heavy	Larger/heavier silhouette.
EnemyCharger	AP_EnemyCharger.prefab	enemies/enemy_charger	Clear forward/charge read.
EnemyTurret	AP_EnemyTurret.prefab	enemies/enemy_turret	Stationary/ranged read.
EnemySplitter	AP_EnemySplitter.prefab	enemies/enemy_splitter	Readable split identity.
EnemyBoss	AP_EnemyBoss.prefab	boss/*	Boss placeholder/variant hook.
Projectile	AP_Projectile.prefab	combat_fx/player_projectile	Small visible player shot.
EnemyProjectile	AP_EnemyProjectile.prefab	combat_fx/enemy_projectile	Enemy shot distinct from player shot.
RoomFloor	AP_RoomFloor.prefab	rooms/floor_tile	Flat tile/plane, does not own collision.
RoomObstacleRock	AP_RoomObstacleRock.prefab	rooms/rock_obstacle	1m blocker visual, bottom at Y=0.
DoorActive/Cleared/Locked/Unavailable	AP_Door*.prefab	doors/*	State-specific door visuals.
RewardPickup	AP_RewardPickup.prefab	rewards/reward_pickup	Generic reward glow/pickup.
BossKeyPickup	AP_BossKeyPickup.prefab	rewards/boss_key	Boss key pickup visual.
HubShop/HubShopCard	AP_HubShop*.prefab	hub/hub_shop*	Shop stand/card visuals.
HubReturnPortal/NextBranchPortal	AP_*Portal.prefab	hub/*portal	Portal visuals; defeated variants may reuse roles.
WeaponMelee/WeaponRanged	AP_Weapon*.prefab	equipment/*weapon	Pickup/display visuals only.
Armor	AP_Armor.prefab	equipment/armor_pickup	Armor pickup/display visual.
ActiveItemPickup	AP_ActiveItemPickup.prefab	items/active_item_pickup	Active item pickup visual.
ConsumableCardPickup	AP_ConsumableCardPickup.prefab	items/consumable_card_pickup	Card pickup visual.
ChestNormal	AP_ChestBasic.prefab	items/Chest	Normal chest. Uses Rafal Chest.fbx and material.
ChestGolden	Current fallback role	items/Chest or future golden folder	Create AP_ChestGolden later if we add distinct prefab binding.

Scale Targets For Common Assets

These are not final art rules; they are safe prototype dimensions that keep the gameplay camera readable.

Entity	Approx Unity Size	Origin / Pivot Rule
Normal chest	About 0.85m wide, 0.55m tall, 1.6m deep for current Rafal chest proportions.	Bottom at Y=0, centered on X/Z.
Coin pickup	0.2m-0.35m visual diameter.	Can float slightly if pickup glow needs it.
Rock obstacle	1m x 1m x 1m visual block.	Bottom at Y=0.
Standard barrel	0.8m diameter, 1m tall.	Bottom at Y=0.
Door	Around 0.8m wide, 1.1m tall, thin depth.	Bottom at Y=0, centered on door port.
Player	Roughly 1.5m-1.8m tall.	Feet/bottom at Y=0.
Normal enemy	0.6m-1.2m depending silhouette.	Grounded enemies bottom at Y=0; flying can float.
Boss	2x-3x player scale depending boss.	Bottom at Y=0 unless flying/burrowing visual requires offset.
Projectile	0.15m-0.3m visual diameter/length.	Centered on projectile runtime object.

Developer Lab Test Sequence

- Open a Developer Lab scene under Assets/_Hollow/Scenes/DeveloperLab to inspect static authoring previews.
- Launch Developer Lab from the profile menu for runtime inspection.
- Use the bottom-right debug button to open the spawn menu if F4 is unreliable on the current platform/keyboard.
- Spawn the asset from its group and compare it to the static lab display.
- If authoring scene and runtime differ, regenerate/export Developer Lab scenes or inspect whether the scene contains a stale copied preview.
- If runtime and prefab differ, inspect PresentationContentCatalog binding and the active AP_* prefab.

Common Problems And Fast Fixes

Symptom	Likely Cause	Fix
Looks correct in FBX preview but wrong in AP prefab	Blender object scale/rotation not applied; wrapper referenced raw mesh only.	Apply transforms in Blender, or bake exact child calibration into AP wrapper and document it.
Invisible in Developer Lab scene but visible from debug spawn	Scene has stale baked preview copy.	Regenerate Developer Lab scenes or replace scene child preview from AP prefab.
Magenta material	Shader/material missing or broken texture path.	Create Unity URP/Lit material and assign local project textures.
Very tiny or huge object	Export scale or unit mismatch.	Use Blender metric units, FBX scale 1, Unity import scale 1. Apply transforms.
Object floats	Pivot/origin or child Y offset wrong.	Bottom of visual mesh should be at Y=0. Grounded runtime host should not need extra offset.
Gameplay collision changes after art swap	Collider/Rigidbody/script was left on visual prefab.	Remove gameplay components. Runtime owns collision.
Old placeholder still appears	Catalog still bound to old AP prefab or scene preview stale.	Check PresentationContentCatalog and regenerate/open lab scene.
Texture works on Rafal machine only	FBX references absolute local texture path.	Put texture inside Unity project and assign it to a Unity material.

Recommended Next Automation

We should add an editor validator that opens every AP_* prefab and reports: renderer exists, bounds non-zero, bottom near Y=0 for grounded roles, root marker exists, missing materials/textures, unsafe colliders/scripts, root scale, and whether the prefab is still fallback. That turns this guide into a one-click acceptance report for Rafal assets.

Hand-off Summary

- Rafal: apply transforms in Blender, export FBX at scale 1, include textures as local project files, and note intended size.

- Martin: replace the visual child inside the existing AP_* prefab, keep marker/catalog stable, test in Developer Lab and debug spawn.

- Both: if a model needs non-uniform calibration, write it down and prefer fixing it in Blender before accepting it as production-ready.